Data Mining and Machine Learning

Gennaro Basile

March 24, 2026

Contents

1 sommario **3**

2 Gradient Descent and Backpropagation **5**

2.1 The Geometry of Optimization 5

2.1.1 The Reality of the Loss Landscape. 5

2.1.2 Different paths toward the minimum. 6

2.1.3 Visualizing the Loss Landscape: Filter Normalization 7

2.2 The Mathematics of Gradient Descent 7

2.2.1 When to Stop Training (Convergence vs. Overfitting) 9

2.3 Stochastic Gradient Descent (SGD) 10

2.3.1 Terminology of SGD 10

1 sommario

- **Backpropagation: concetti fondamentali**
  - Introduzione alla sezione sul backpropagation algorithm (slide 19)
  - Definizione, ruolo storico e collegamento con automatic differentiation (slide 20)
  - Forward phase e backward phase nelle reti neurali viste come computational graphs (slide 21)
- **Notazione matematica per la backpropagation**
  - Attivazioni dei neuroni in forma scalare e matriciale (slide 22)
  - Weighted input z^l, relazione tra z^l e a^l (slide 23)
  - Assunzioni sulla cost function necessarie per applicare backpropagation (slide 24)
  - Giustificazione della prima assunzione e passaggio al singolo training example (slide 25)
  - Esempio con quadratic cost function e verifica delle assunzioni (slide 26)
- **Le quattro equazioni fondamentali della Backpropagation**
  - Introduzione alla sezione sulle quattro equazioni fondamentali (slide 27)
  - Definizione dell'errore del neurone δ_j^l (slide 28)
  - BP1: errore nello strato di output (slide 29)
  - Forma matriciale di BP1 e caso della quadratic cost (slide 30)
  - Dimostrazione di BP1 tramite chain rule (slide 31)
  - BP2: propagazione dell'errore allo strato precedente (slide 32)
  - BP3: derivata della cost rispetto ai bias (slide 33)
  - BP4: derivata della cost rispetto ai pesi (slide 34)
  - Riassunto compatto delle quattro equazioni BP1–BP4 (slide 35)
- **Implicazioni della Backpropagation e dinamica dell'apprendimento**
  - Conseguenze pratiche delle equazioni BP: neuroni saturi, gradienti piccoli, apprendimento lento (slide 36)
  - Algoritmo completo della backpropagation passo per passo (slide 37)
- **Combinare Gradient Descent e Backpropagation**
  - Introduzione alla sezione sulla combinazione GD + BP (slide 38)
  - Procedura pratica per combinare feedforward, backpropagation e gradient descent su mini-batch (slide 39)
- **Interpretazione intuitiva della Backpropagation e problemi dei gradienti**
  - Logica intuitiva della BP: come una piccola variazione di un peso influenza la cost (slide 40)
  - Propagazione delle variazioni attraverso le attivazioni del layer successivo (slide 41)
  - Somma dei contributi lungo tutti i possibili cammini nella rete (slide 42)
  - Espansione della chain rule lungo un intero percorso dalla weight alla cost (slide 43)
  - Vanishing gradient problem e impatto sull'aggiornamento dei pesi (slide 44)
  - Altri problemi dell'addestramento profondo: exploding gradients, difficoltà di convergenza, minimi spurii (slide 45)
- **Esercizi finali**
  - Esercizi su dimostrazione di BP2, BP3, BP4 e varianti della backpropagation (slide 46)

2 Gradient Descent and Backpropagation

2.1 The Geometry of Optimization

Once a neural network makes a prediction and calculates the error (Loss/Cost), it must adjust its weights to improve. The algorithm used to find the optimal weights is called **Gradient Descent**.

Gradient Descent

Gradient Descent is an iterative optimization method used to minimize a function by moving the parameters step by step in the direction opposite to the gradient.

To understand Gradient Descent, we must visualize the problem geometrically. Imagine the **Cost Function** as a physical landscape (a hypersurface) where the coordinates are the model's weights, and the elevation is the error.

- **High elevation:** High error (bad predictions).
- **Low elevation (Valleys):** Low error (good predictions).

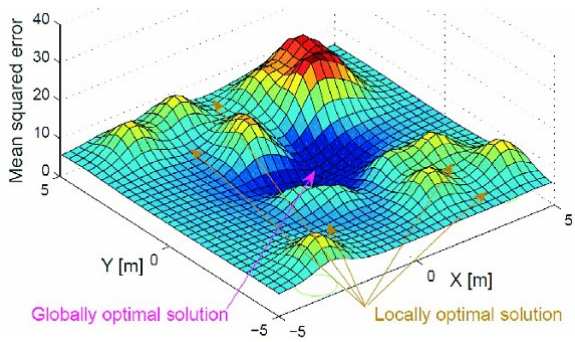

Figure 2.1.1: *The Cost Hypersuperface*

Cost Hypersurface

A cost hypersurface is the geometric representation of the cost function in the parameter space of the model. Each point of this space corresponds to one specific configuration of the model parameters.

The Goal of Training: To navigate the network's weights down the landscape until they reach the **Global Minimum** (the absolute lowest possible error).

Global Minimum

The global minimum is the absolute lowest point of the cost function over the entire parameter space.

Local Minimum

A local minimum is a point whose cost is lower than that of nearby points, but not necessarily the lowest over the entire parameter space.

2.1.1 The Reality of the Loss Landscape.

In simple models (like Linear Regression), this landscape is a smooth, bowl-shaped **Convex Function** with only one perfect bottom.

Convex Function

A convex function is a function whose surface has only one global minimum and no local minima distinct from it.

However, in Deep Learning (with millions of parameters), the landscape is highly **Non-Convex**. It is filled with jagged peaks, flat plateaus, and **Local Minima** (shallow valleys that trap the model before it reaches the true bottom).

Non-Convex Function

A non-convex function is a function whose surface may contain many local minima, local maxima, saddle regions, and irregular shapes.

2.1.2 Different paths toward the minimum.

The path followed during optimization depends on many factors. The goal is to find the weight configuration that minimizes the cost function, but convergence does not always happen in the same way.The speed of convergence depends on:

- the starting point,
- the shape of the cost surface,
- the step size used during exploration.

The figure (2.1.2) shows two different behaviors. In one case, the path is well-behaved and quickly reaches the minimum. In the other case, the path oscillates inside a narrow valley and converges more slowly.

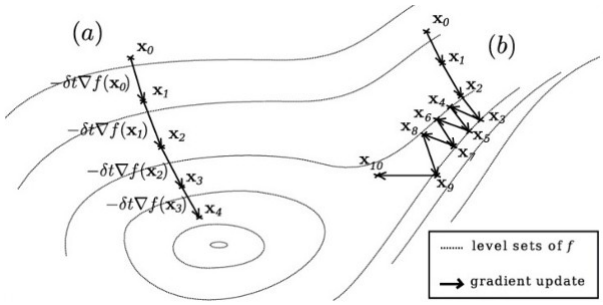

Figure 2.1.2: *Different path for the minimum*

This tells us that optimization is not only about the formula, but also about the geometry of the problem. A bad learning rate or a difficult landscape can make training unstable or slow.

Convergence

Convergence is the process through which an iterative optimization algorithm approaches a stable solution, ideally a minimum of the cost function.

In practice, convergence means that after several updates the parameters are no longer changing significantly, and the cost function does not improve much anymore.

Researchers use advanced projection techniques, like *Filter Normalization*, to properly visualize these massive, chaotic multidimensional landscapes in 3D to understand how models learn.

2.1.3 Visualizing the Loss Landscape: Filter Normalization

Neural network loss landscapes exist in millions of dimensions. To visualize them, we plot a 2D slice as a 3D mountain range (where height = error). However, doing this naively creates a problem.

- **The Problem (Scale Invariance):** In modern networks, you can scale weights up and down mathematically without changing the final prediction. For example:
  - we multiply the weights of one layer by a constant,
  - and divide the weights of the following layer by the same constant.

  Plotting these raw, unadjusted weights creates optical illusions, making smooth, easily trained valleys look like chaotic, jagged cliffs.
- **The Solution (Filter Normalization):** To reveal the *true* geometry of the landscape, we must scale our X and Y graphing axes to perfectly match the actual size (norm) of the model's filters.

The 5-Step Visualization Process

1. **Center:** Start at the fully trained model's parameter vector (θ^*).
2. **Axes:** Pick two random direction vectors (δ and η) to act as your X and Y axes.
3. **Normalize:** Rescale δ and η so they match the scale of the model's filters.
4. **Grid Evaluation:** Shift slightly around the center point and calculate the Loss across a 2D grid using the formula:
$$\theta = \theta^* + x\delta + y\eta$$
5. **Plot:** Map these Loss values to the Z-axis (height) to create an accurate 3D surface map.

Filter Normalization

Filter normalization is a visualization technique in which the directions used to explore the loss landscape are rescaled according to the norm of the model filters, so that the plotted surface is not distorted by scale invariance.

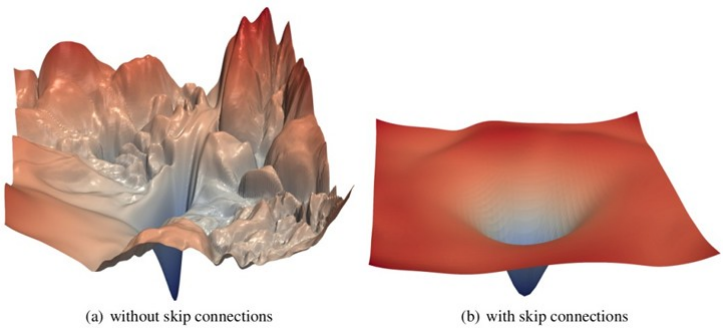

Figure 2.1.3: *Filter Normalization*

In short, filter normalization helps us visualize the true shape of the loss landscape around the final trained model, avoiding misleading distortions caused by the scaling of the weights.

2.2 The Mathematics of Gradient Descent

We defined Gradient Descent as a numerical method used to find the minimum of a function of many variables. In this context, the function is the cost function C, and the variables are the weights of the model.

Cost Function

The cost function is a function that measures how far the predictions of a model are from the desired targets over the training data.

The cost is computed over the entire training dataset at each iteration. We can write the approximate variation of the cost:

$$\Delta C \sim \sum_{i=1}^{n} \frac{\partial C}{\partial w_i} \Delta w_i,$$

where n is the number of weights.

This formula is the first-order approximation of how the cost changes when the weights are perturbed. It is a multivariable version of the linear approximation from calculus.

Partial Derivative

A partial derivative measures how a multivariable function changes when only one variable is varied while all the others are kept fixed.

In optimization, the partial derivative $\frac{\partial C}{\partial w_i}$ tells us how sensitive the cost is to changes in the weight w_i.

Link between ΔC and Δw. We can rewrites the previous expression using vector notation and introduces the gradient.

Gradient

The gradient of a function is the vector containing all its partial derivatives. It points in the direction of the greatest increase of the function.

It is possible states that

$$\Delta C \sim \nabla C \cdot \Delta w,$$

where

$$\nabla C = \left(\frac{\partial C}{\partial w_1}, \ldots, \frac{\partial C}{\partial w_n} \right)^T$$

is the gradient column vector, and Δw is the vector of weight variations.

The dot product $\nabla C \cdot \Delta w$ tells us how the cost changes when we move in the direction Δw. If we want the cost to decrease, we should choose Δw so that this dot product becomes negative. we introduce a small positive parameter η and pose that:

$$\Delta w = -\eta \nabla C,$$

where $\eta > 0$ is a small parameter. Substituting this into the previous formula gives:

$$\Delta C \sim -\eta \nabla C \cdot \nabla C = -\eta \|\nabla C\|^2. \tag{1}$$

Since $\|\nabla C\|^2 \geq 0$ and $\eta > 0$, we obtain $\Delta C < 0$ whenever the gradient is nonzero. This guarantees that, locally, the cost decreases.

Dot Product

The dot product between two vectors measures how much one vector points in the direction of the other.

Learning Rate

The learning rate is a positive scalar that controls the size of the update step during optimization.

This is the central mathematical intuition of Gradient Descent: move in the opposite direction of the gradient because that is the direction of the steepest local decrease.

This gives us the fundamental **Weight Update Rule** of Machine Learning:

$$w_{\text{new}} = w_{\text{old}} - \eta \nabla C \tag{2}$$

My new weight equals my old weight, minus a small step in the direction that reduces the error.

The Danger of the Learning Rate (η). The Learning Rate is arguably the most important hyperparameter in Deep Learning. It dictates how large of a step the algorithm takes down the mountain and on the geometry of the cost function.

- **If η is too small:** convergence may be very slow.
- **If η is too large:** the algorithm may overshoot the minimum and oscillate.

So the elegance of Gradient Descent lies in its simplicity, while its practical difficulty lies in choosing appropriate hyperparameters and handling difficult optimization landscapes.

2.2.1 When to Stop Training (Convergence vs. Overfitting)

Training should not continue indefinitely. We monitor the Cost (Loss) Function during training to decide when to stop:

1. **Convergence:** Stop when the loss function reaches a flat plateau and stops decreasing. The model has learned all it can.
2. **Early Stopping (Preventing Overfitting):** We must monitor the loss on both the *Training Set* and the *Validation Set*. If the Training Loss continues to drop, but the Validation Loss starts to *increase*, the model is **Overfitting** (memorizing the training data instead of learning general patterns). Training must be stopped immediately.

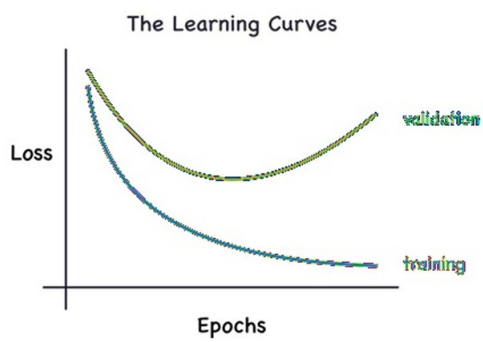

Figure 2.2.1: *Loss Validation vs Training set*

In traditional data science, having too many dimensions (features) is bad: it causes data sparsity and increases compute time (The Curse of Dimensionality).

However, in Deep Learning optimization, high dimensionality is actually a blessing. In a 3D landscape, it is easy to get stuck in a valley (Local Minimum) because there are only two directions to move (X and Y). In a 1-million-dimensional landscape, if 999,999 directions go

uphill, there is almost certainly at least one direction that still goes downhill, allowing the algorithm to escape the trap and continue learning.

2.3 Stochastic Gradient Descent (SGD)

Standard Gradient Descent has a massive computational flaw: to take a *single* step down the mountain, it must calculate the error across the *entire* training dataset. If your dataset has 10 million images, computing 10 million errors before updating the weights once is too slow.

The solution is **Stochastic Gradient Descent (SGD)** using **Mini-Batches**. Instead of looking at the whole dataset, SGD grabs a small, random handful of data (a mini-batch, e.g., 32 or 64 images). It calculates the gradient on just that small batch, and updates the weights immediately.

Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is an optimization method in which the gradient is estimated using only a small randomly selected subset of the training data instead of the entire dataset.

- **Trade-off:** The gradient calculated on a mini-batch is *noisy* and imprecise compared to the exact gradient of the whole dataset. The path down the mountain will be jagged and zig-zagged.
- **Benefit:** Because it updates the weights instantly after just 32 images, the model trains exponentially faster.

Mini-batch update algorithm, batch, and epoch The practical algorithm used in mini-batch learning can be summerized like:

- `Step 1`: select m samples to form a mini-batch.
- `Step 2`: compute the gradient approximation on that mini-batch.
- `Step 3`: update the weights and biases using

$$w'_k = w_k - \frac{\eta}{m} \sum_j \frac{\partial C_{x_j}}{\partial w_k}, \qquad b'_l = b_l - \frac{\eta}{m} \sum_j \frac{\partial C_{x_j}}{\partial b_l}.$$

- `Step 4`: choose another random mini-batch and repeat the process.
- `Step 5`: continue until the entire training set has been processed.
- `Step 6`: repeat the whole process for multiple passes until convergence is reached.

2.3.1 Terminology of SGD

- **Mini-Batch:** The small subset of data used for a single weight update.
- **Iteration (or Step):** One update of the model's weights based on one mini-batch.
- **Epoch:** One complete pass through the *entire* training dataset. (If your dataset has 1,000 images and your batch size is 100, it takes 10 iterations to complete 1 Epoch).